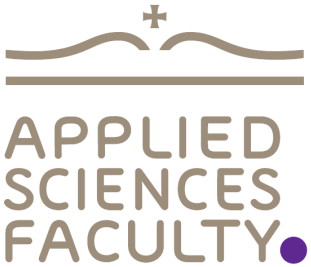
C++ — лекція 16. Перетворення типів





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Коротко про перетворення типів

- Можна попросити компілятор перетворити змінну одного типу в змінну іншого.
- Частину перетворення типів компілятор виконує неявно (наприклад, short до int). Про решту йому слід казати явно, (наприклад про перетворення `void*` до `*double`).
- Перетворення від менш ємного до більш ємного типу, наприклад short до int чи float до double, називається просуванням, і є дещо м'якшим перетворенням, ніж власне перетворення.
- Компілятор намагатиметься зберегти вміст змінної, поки це можливо. Зокрема, якщо приводити більший тип (скажімо, int), до меншого (скажімо, char), то якщо значення може поміститися — його буде перенесено коректно[1].
- C++, на додачу до запозиченого з C, має свої, більш просунуті засоби перетворення типів.

[1] Якщо ні — результат невизначений.

Перетворення в стилі С:

```
void *void_ptr = .....;
double *d_ptr = (double *)void_ptr;

int n;     // Краще взяти не int, а intptr_t ––– щоб точно
           // вистачило для вказівника!
int *ptr;
n = (int)ptr;
```

Але і таке є можливим:

```
const char *c="ABCD";
n = (int)*c;
cout << n << endl; // Друкує ASCIIкод– літери 'A'
                   // платформозалежний( результат!)
n = 33;
c = (char*)&n;
cout << c << endl; // Друкує '!' платформозалежний( результат!)

n = 1233333;
c = (char*)&n;
cout << c << endl; // Друкує всілякі спецсимволи
```

Перетворення в стилі C – основні недоліки

- Візуально наявність перетворення не дуже помітна в коді.
- Немає явного контролю, як саме здійснюється перетворення.
- Тому потрібно уникати їх використання!
- ◇ GCC та Clang можна попросити повідомляти про їх використання опцією `-Wold-style-cast`.

Перетворення в стилі C++:

- **static_cast** — перетворення типів, для випадків, коли компілятор знає, як змінну одного типу перетворити у змінну іншого:

```
d_ptr5 = static_cast<double *>(ptr_without_type);
```

- **reinterpret_cast** — команда компілятору вважати змінну одного типу такою, що має інший. Небезпечне і непортабельне перетворення!
- **const_cast** — перетворення, що прибирає, чи додає модифікатор const.
- **dynamic_cast** — робота із (поліморфними) деревами успадкувань

Такі перетворення видно здалеку, програміст чітко конкретизує свій намір.
Крім того, їх складніше писати. В даному випадку це — перевага! Перетворення типів має бути надзвичайним заходом, а не буденністю.

static_cast — 1

Нестрого кажучи, перетворення типів із максимальним збереженням інформації.

- перетворення, якщо існують явні чи неявні перетворення між двома типами:

```
int n = static_cast<int>(3.14);

class A{
    .....
    explicit A(int x);
};
A a = static_cast<A>(10);
```

- Перетворення вверх по ієрархії успадкування[2]:

```
class B{ ... };
class D: public B { ... };

B b;
D d;
B &br = d; // OK – підстановка, посилання щоб не було зріз-
ки
D& dr = static_cast<D&>(br); // Перетвоюємо назад
```

static_cast — 2

- Перетворення між перерахуваннями та int:

```
enum E1{RED1, GREEN1, BLUE1};
enum E2{RED2, GREEN2, BLUE2};

    E1 v1 = RED1;
    E2 v2 = BLUE2;
    int  i  = 1;
    v1 = static_cast<E1>(i);
    v2 = static_cast<E2>(v1);
```

- Перетворення вказівників void* у вказівники на певний тип.
- **УВАГА!** Перетворювати між вказівниками на різні типи не вміє!
- Явне нехтування повернутого значення:

```
static_cast<void>(obj.get_size());
```

- Деякі інші аналогічні перетворення.
- Не може використовуватися для маніпуляції модифікаторами const і volatile.

reinterpret_cast — 1

reinterpret_cast — насильство над компілятором. Команда вважати змінну одного типу змінною іншого. Дуже небезпечний засіб!

Можливі (повторимося — потенційно небезпечні!) використання:

- Перетворення між цілими типами та вказівниками.
- Перетворення між вказівниками на непов'язані типи.
- Серіалізація.
- Однак, пам'ятайте про strict aliasing!

reinterpret_cast — 2

```
int si0=0x12345678; // 8 вих16— цифр — чотири байти
char *ss0=reinterpret_cast<char*>(&si0);
for(int i=0; i<4; ++i)
{
    cout << hex << static_cast<int>(ss0[i]) << ", ";
}
cout << endl;
```

Можливий результат:

```
78, 56, 34, 12,
```

- Причина: система є little endian, як x86. Буває також big endian – там би було 12, 34, 56, 78.
- Перевірити під час компіляції можна за допомогою std::endian із C++20, <bit>. вміст...

```
if constexpr (std::endian::native == std::endian::big)
        std::cout << "big-endian\n";
    else if constexpr (std::endian::native == std::endian::little)
        std::cout << "little-endian\n";
    else std::cout << "mixed-endian\n";
```

Strict aliasing

Якщо зовсім коротко, компілятор вважає, що вказівники різних типів вказують на різні області пам'яті, за природними винятками:

- Вказівники на типи із різними комбінаціями модифікаторів signed/unsigned, const/volatile, та іншими сумісними типами, можуть вказувати на ту ж пам'ять.
- Вказівники на типи можуть вказувати всередину структур та об'єднань, що містять поля відповідних типів.
- До C++17: char*, signed char*, unsigned char* можуть вказувати будь-куди (для роботи із "сирою" пам'яттю).
  - **З C++17**: безпосередньо звертання до такого масиву char, зробивши reinterpret_cast до типу, який там реально зберігається – UB.
  - Щоб його використовувати, потрібно скористатися std::launder() – див. далі.
- Вказівники на базові класи можуть вказувати на нащадків (природно!).
- "Common initial sequence" – якщо структури, що входять в union, починаються з сумісних членів даних, до них можна звертатися через будь-яку з структур.
  - Це дозволяє, наприклад, тип структури зберігати безпосередньо в union, не виносячи назовні.

Зокрема:

- Навіть якщо два різних типи структур оголошені однаково, компілятор вважає, що вказівник на екземпляр однієї із них, ніколи не вказуватиме на другу.

Type Punning

- Strict aliasing – важлива для оптимізацій вимога[3], однак вона заважає інтерпретувати вміст змінної одного типу як такий, що має інший тип.
- Це іноді потрібно. Постає питання, а як?
- Reinterpet_cast – UB.
- ◇ Крім того, навіть якби була можливість пере-інтерпретовувати вказівники, якщо вказівник вирівняно не так, як потрібного для конкретного типу – це UB[4].
- Стандартний метод – memcpy.

```
double d;
int64_t i;
std::memcpy(&i, &d, sizeof(d));
```

- ◇ Втрати продуктивності мають шанс бути не такими жахливими, як можна очікувати – сучасні компілятори розпізнають цю ідіому. Іноді навіть для масивів змінних.

[3] Не те щоб Лінус Торвальдс був з цим згоден...

[4] Тобто, якщо для int "природне" вирівнювання вашого компілятора 4, то навіть якщо якоюсь магією ви отримали коректний вказівник на коректний int, вирівняний на 2, але не на 4 – звертатися до нього це UB.

std::bit_cast (C++20)

- Оскільки ці "танці" з memcpy доволі неестетичні, в C++20 додали спеціалізований засіб:

```
template< class To, class From >
constexpr To bit_cast( const From& from ) noexcept;
```

- Отримати значення типу To, яке є відповідною інтерпретацією бітів From.
- Якщо в To є біти вирівнювання (padding) – вони невизначені.
- Якщо відповідного значення для To не існує – невизначена поведінка.
- Якщо значень кілька – котре буде використано, не вказується.
- Оскільки копіює, не має проблем із strict aliasing.
- Обговорення потрібності: `https://stackoverflow.com/a/53402217`.
- Не вміє (поки?) працювати з масивами[5].
- Щоб обійти, можна використати трюк:

```
struct uint32_chars {
        unsigned char arr[sizeof( uint32_t )] = {} ;
};

uint32_t data = .....
uint32_chars f;
std::memcpy( f.arr, &data, sizeof(data));
unsigned int result = bit_cast<uint32_t>(f);
```

[5] Наприклад, масив 8 char та double.

Strict aliasing – посилання

- `http://dbp-consulting.com/tutorials/StrictAliasing.html`
- `http://cellperformance.beyond3d.com/articles/2006/06/understanding-strict-aliasing.html`
- `https://vector-of-bool.github.io/2022/05/11/char8-memset.html` – приклад, коли робота із u8string оптимізується до використання memset(), в string – ні.
- `https://www.qt.io/blog/2011/06/10/type-punning-and-strict-aliasing`
- "What is the Strict Aliasing Rule and Why do we care?: OR Type Punning, Undefined Behavior and Alignment, Oh My!".
- "A bug story: data alignment on x86" – приклад про важливість вирівнювання, навіть якщо type punning "вдався".
- "When LICM fails us" та "Aliasing" – приклад впливу альясингу.

const_cast

Засіб маніпуляції модифікаторами const і volatile.

```
int var = 3;
const int& const_ref_var = var;
const_cast<int&>(const_ref_var) = 4;
// OK ––– модифікує неконстанту– змінну var

const int var_const = 5;
int *vptr = const_cast<int*>(&var_const);
*vptr = 4; //!!! Undefined behavior

void print (char* str){
    cout << str << endl;
}
const char* s = "ABCD";
print( const_cast<char*>(s) ); // Обхід некоректної сигнатури
```

- "const_cast: A Necessary Evil" – приклад, коли, станом на C++26, const_cast – обов'язковий.
- Детальніше див. "Cppref: const_cast.

dynamic_cast – 1

- Працює тільки із вказівниками та посиланнями.
- Якщо клас поліморфний, перетворення виду:

  Base *pb = ...;
  Derived* pd = **dynamic_cast**<Derived*>(pb);

  поверне коректний вказівник на Derived, якщо pb справді вказує на об'єкт класу-потомка, і **нульовий вказівник** в протилежному випадку.
- Для посилань, так як вони нульовими не бувають, за неможливості перетворення генеруватиметься виключення `std::bad_cast`.
- Перетворення від вказівника чи посилання на потомок у вказівник чи посилання на предка діє завжди, як і відповідне неявне перетворення – не залежно від того, поліморфний клас, чи ні.

dynamic_cast – 2

- Для роботи із поліморфними типами, потребує RTTI — Run-Time Type Information. Для його коректної роботи ця можливість повинна бути дозволеною.
- Таке перетворення заборонене, якщо клас не поліморфний — не містить жодної віртуальної функції. Тому такий код не скомпілюється:

```
struct Base {};
struct Derived : Base {};
        .....
Base* ptr = new Derived;
Derived* d = dynamic_cast<Derived*>(ptr);
```

  Справді, для таких класів працювати із потомками за допомогою вказівників на базовий клас не можна.
- `dynamic_cast<void*>` повертає вказівник на найбільш пізнього нащадка. Працює і для множинного успадкування.
- Детальніше див. "cppreference: dynamic_cast".
- Деякі подробиці детально описано тут: "Dynamic_cast and typeid as (non) RTTI tools".
- "Once more about dynamic_cast, a real use case".

- В термінах C++ перетворення C виглядає як послідовні спроби зробити щось таке:
- const_cast
- static_cast
- const_cast, після якого static_cast
- reinterpret_cast
- const_cast, після якого reinterpret_cast

Посилання

- "Implicit string conversions to booleans" – приклад небезпеки неявних перетворень.
- "Why does INVALID_HANDLE_VALUE cast through a LONG_PTR first?"

typeid

- Оператор typeid дозволяє працювати із типом виразу.
- Він повертає змінну типу type_info (який визначено у файлі <typeinfo>). Такі змінні можна порівнювати за допомогою операторів == та !=. За допомогою методу name() її можна перетворювати у стрічку, що описує тип.

```
int *pv,v;
cout << "pv type is: " << typeid(pv).name() << endl;
cout << "v  type is: " << typeid(v).name() << endl;
```

  під VS-2010 виведе:

```
pv type is: int *
v  type is: int
```

- **УВАГА!** Конкретний вміст стрічки залежить від компілятора! Приклад gcc-4.6:

```
pv type is: Pi
v  type is: i
```

- Якщо RTTI дозволено, воно використовується оператором typeid для визначення конкретних типів поліморфних класів.
- Застосований до вказівників, він поверне тип вказівника. Однак, застосований до самих об'єктів, поверне реальний їх тип.

Приклад typeid з поліморфним класом

```
#include <typeinfo>
    .....
    class Base {
        virtual void f(){}
    };

    class Derived : public Base {};

    Base* b = new Base;
    Base* d = new Derived;
    cout << "b is: " << typeid(b).name() << endl;
    cout << "d is: " << typeid(d).name() << endl;
    cout << "*b is: " << typeid(*b).name() << endl;
    cout << "*d is: " << typeid(*d).name() << endl;
```

З-під Visual Studio-2010, виведе:

```
b is: PAVBase@?5?main@
d is: PAVBase@?5?main@
*b is: ?AVBase@?5?main@
*d is: ?AVDerived@?6?main@
```

Часте використання typeid свідчить про погано спроектовану програму!

Бажано користуватися механізмом поліморфізму, а не явними маніпуляціями з типом.

- typeid() працює і не для поліморфних типів.
- Для поліморфних типів typeid() обчислюватиме свій аргумент.
- Для не поліморфних – ні. Тоді йому можна передавати й ім'я типу.

```
auto& info1 = typeid(int);
```

- Чи й так:

```
struct B{
    B(){ std::cout << "Constructing B" << std::endl; }
    int f(){ std::cout << "B::f()" << std::endl; return 1; };
};

    auto& info2 = typeid(B);
    std::cout << info2.name() << std::endl;
    auto& info3 = typeid(B());
    std::cout << info3.name() << std::endl;
```

- Насправді, typeid(5) виходить еквівалентним до typeid(decltype(5)).
- Так як вираз не обчислюється, використання нульового вказівника не буде UB:

```
    auto& info = typeid(*((B*)nullptr));
    std::cout << info.name() << std::endl;
```

- Більше того, для поліморфного типу теж можна передати нульовий вказівник – це ще не UB:

```
try {
    auto &info4v = typeid(*((Dv *) nullptr ));
    std :: cout << info4v.name() << std::endl;
}catch(std :: exception& w){
    std :: cerr << "Exception: " << w.what() << std::endl;
}
```

  Буде просто кинуто виключення bad_typeid.

- Але ось таке вже є невизначеною поведінкою! Фокус вище діє лише за безпосереднього застосування до нульового вказівника. Якщо нульовий вказівника глибше у виразі – UB.

```
try {
    auto &info5v = typeid(((Dv *) nullptr )->f());
    std :: cout << info5v.name() << std::endl; //! UB!
}catch(std :: exception& w){
    std :: cerr << "Exception: " << w.what() << std::endl;
}
```

- typeid() відрізняє вказівник на функцію від самої функції, і вказівник від масиву – звичні правила "розкладання"–decay не діятимуть.
- Найбільш зовнішня константність не впливає на тип:

  typeid(int) == typeid(const int);
  typeid(int*const) == typeid(int*);

  Однак, більш вкладені дають різні типи:

  typeid(int const*) != typeid(int*);

- Детальніше див. `https://blog.panicsoftware.com/dynamic_cast-and-typeid-as-non-rtti-tools/` – звідки взято приклад вище та ідеї прикладів із попередніх слайдів.

Час життя об'єктів

- Етапи життя об'єкта: до виклику конструктора, під час цього виклику, після виклику конструктора, під час виклику деструктора, після нього.
- Для об'єктів із тривіальною ініціалізацією, час життя розпочинається з моменту виділення пам'яті.
  - Аналогічно, тривіальність присвоєння означає, що присвоювати можна, копіюючи побітово.
  - Тривіальне переміщення – аналогічно, що потрібно скопіювати побітово, і більше не звертатися до вихідних даних.
- За наявності нетривіальних конструкторів, час життя об'єкта розпочинається після виклику конструктора.
- Аналогічно, час життя тривіальних об'єктів припиняється в момент звільнення чи повторного використання пам'яті, виділеної для них.
- Для об'єктів з нетривіальною ініціалізацією – на початку виклику деструктора.
- Після виділення пам'яті, але до завершення конструктора, можливе обмеження використання об'єкту – власне, конструктор працює саме в таких умовах, але потрібно бути акуратними.
- Поведінка не сконструйованого до кінця об'єкту може відрізнятися.
  - Наприклад, у конструкторі, віртуальні функції викликаються як не віртуальні.
- Окрім як в конструкторі – де звертання відбувається (навіть якщо неявно) за допомогою this, використовувати вказівник чи посилання на виділену пам'ять об'єкту, час життя якого ще не розпочався, можна використовувати лише як вказівник `void*` або вказівник на char чи byte.

- Якщо час життя об'єкту було припинено, але без звільнення пам'яті, розпочато у ній час життя нового об'єкту того ж типу, вказівники та посилання на нього залишаються валідними, за умови:
  - ◇ об'єкт не був константою і не містив константних членів.
- Якщо ця умова не виконується, "правильний" вказівник на новий об'єкт можна отримати за допомогою std::launder().
  - ◇ Крім випадку, коли цілий об'єкт був оголошений як константа – тоді перевикористання його пам'яті є невизначеною поведінкою.
- Приклад із стандарту[6]:

```
struct C {
  int i;
  void f();
  const C& operator=( const C& );
};

const C& C::operator=( const C& other) {
  if ( this != &other ) {
    this->~C();                 // lifetime of *this ends
    new (this) C(other);        // new object of type C created
    f();                        // well-defined
  }
  return *this;
}
```

[6] https://timsong-cpp.github.io/cppwp/n4659/basic.life#8.

std::launder() – C++17

- std::launder() повідомляє компілятору, що об'єкт змінився на інший, у випадку, коли оптимізатор компілятора міг покладатися на це знання.
  - ◇ Приклад 1 – об'єкт містить члени-константи, які не можуть змінюватися за час життя об'єкту.
  - ◇ Приклад 2 – об'єкт-нащадок замінено на об'єкт типу предка.
  - ◇ Це, зокрема, впливає на девіртуалізацію – vptr теж вважається константою.
- Через зміни в C++17, використовувати char* чи аналоги – масив байт, для доступу до об'єкту іншого типу стало невизначеною поведінкою – потрібно використовувати std::launder():

```
struct Y { int z; };
alignas(Y) std::byte s[sizeof(Y)];
Y* q = new(&s) Y{2};
const int f = reinterpret_cast<Y*>(&s)->z; // Class member access is undefined
                                           // behavior: reinterpret_cast<Y*>(&s)
                                           // has value "pointer to s" and does
                                           // not point to a Y object
const int g = q->z; // OK
const int h = std::launder(reinterpret_cast<Y*>(&s))->z; // OK
```

Приклад з `https://en.cppreference.com/w/cpp/utility/launder`.

- На strict aliasing std::launder не впливає.

Використання std::launder()

- Враховуючи описане на попередньому слайді, std::launder потрібна, якщо:
  - ◇ Отримати вказівник на новий об'єкт, створений замість старого у тій же пам'яті, якщо об'єкт має константні члени, або новий об'єкт має фактичний тип предкового для вихідного.
  - ◇ Отримати валідний вказівник на об'єкт, створений у "сирій" виділеній пам'яті за допомогою placement new.
- Увага, std::launder не створює об'єкт (на відміну від std::start_lifetime_as), лише дозволяє звернутися до об'єкту, що вже існує.
- В C++26 дозволили reinterpret_cast після placement new, замість launder:

```
alignas(T) std::byte storage[sizeof(T)];
::new (&storage) T();
// ...
T *ptr_ = reinterpret_cast<T*>(&storage);  // UB before C++26
```

Все рівно такі конструкції широко використовуються.

Уточнення щодо часу життя об'єктів в C++20

- В C++17 і раніше, такий код – невизначена поведінка:

```
struct S { int a, b; };
    .............

S* p = static_cast<S*> ( malloc(sizeof(S)) >;
p->a = 1;   // Невизначена поведінка до C++20
```

  оскільки, формально, об'єкта тоді ще не існує – malloc не знає про конструктори.
- C++20 уточнив правила "неявного" початку життя об'єктів. Так може починатися життя:
  - об'єктів скалярних (фундаментальних) типів,
  - їх масивів та інших агрегатів,
  - класів з відповідними тривіальними конструкторами та деструкторами.
- Почати час життя цих типів "неявно" можуть:
  - операції, що розпочинають час життя масивів (unsigned/signed) char та std::byte,
  - оператори new та new[],
  - std::allocator<T>::allocate(),
  - функції виділення пам'яті C – malloc(), calloc(), realloc(), aligned_alloc() тощо,
  - memcpy() та memmove(),
  - std::bit_cast.
- В принципі, це лише узаконення поширеної практики.

Уточнення щодо припинення життя об'єктів в C++20

- Щоправда, разом з уточненням з попереднього слайду, тепер виклик деструкторів для таких типів "справді" припиняє його життя:

```
using T = int;
int x;
x.~T();
x = 5; // Невизначена поведінка.
```

std::start_lifetime_as() та std::start_lifetime_as_array() з C++23

- Функції std::start_lifetime_as() та std::start_lifetime_as_array() з <memory> вирішують багато проблем зміни типу об'єктів.
- Вони дозволяють повідомити компілятору, що блок пам'яті слід інтерпретувати як об'єкт певного типу:

```
struct someinfo_t {
  int version;
  unsigned type;
      ..............
};

void process_data(std::span<std::byte> read_data) {
     const auto* someinfo = std::start_lifetime_as<someinfo_t>(
          read_data.data()
       ......................
}
```

- Розпочинає життя об'єкту, (на відміну, від std::launder()), хоч і не виконує жодного коду
- Детальніше див. оригінальний документ: "Explicit lifetime management".
- Есей на тему: "Messing with lifetime"

Посилання

- https://blog.panicsoftware.com/objects-their-lifetimes-and-pointers/ – дуже важлива стаття!
- https://en.cppreference.com/w/cpp/utility/launder
- https://stackoverflow.com/a/70419156 – відповідь на запитання "What is the purpose of std::launder?".
- https://stackoverflow.com/questions/62114622/stdlaunder-use-cases-in-c20.
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0532r0.pdf – "On launder()".
- https://timsong-cpp.github.io/cppwp/n4659/basic.life#8 – "Object lifetime".
- https://eel.is/c++draft/basic.memobj#intro.object-10 – "*implicitly creating objects*".
- https://miyuki.github.io/2016/10/21/std-launder.html – старенька стаття, яка гуглиться по темі.
- https://habr.com/ru/articles/540954/ – "C++17. Функция стандартной библиотеки std::launder и задача девиртуализации"
- "The C++ Object Lifecycle" – цікавий есей на тему.
- "Reminder: When a C++ object fails to construct, the destructor does not run".
- "The correct way to do type punning in C++ - The second act".
- "What reinterpret_cast doesn't do" – про роль start_lifetime_as().

